

Two heuristics for one of Bin-Packing problems [★]

Barashov Egor ^{*,**} Grishin Egor ^{*,**} Lemtuzhnikova Daria ^{*,***}

^{*} V. A. Trapeznikov Institute of Control Sciences of Russian Academy of Sciences 65 Profsoyuznaya street, Moscow 117997, Russia (e-mail: barashov.eb@gmail.com).

^{**} Lomonosov Moscow State University, GSP-1, Leninskie Gory, Moscow, 119991, Russia (e-mail: grishin.em16@physics.msu.ru).

^{***} Moscow Aviation Institute, 4 Volokolamskoe highway, Moscow, 125993, Russia (e-mail: darabbt@gmail.com).

Abstract: In this article, a new practical problem is proposed, as well as algorithms for solving the one-dimensional bin packing problem. The generalization of this problem is one of the most fundamental problems of combinatorial optimization and has been widely studied for decades. Our formulation for this problem takes into account not only the different weights of products and separability but the difference in their types. An objective function is formulated that minimizes the sum of the components with weights responsible for different characteristics of product distribution. Parameter generation of the problem is based on data that approximate the real one. In order to compare the algorithms 168 test instances were generated. In addition they were solved optimally using the Gurobi solver with a time limit of 1 hour. The algorithms proposed are based on the separation of the set of subjects under consideration on the basis of divisibility. Also, the proposed algorithms have been tested on a large family of generated instances with the number of bins from 100 to 1000 ones.

Copyright © 2022 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: bin-packing problem, cargo transportation, mathematical programming, heuristic.

1. INTRODUCTION

The bin-packing problems are actively used in modern technological optimization problems. The bin-packing formulation is one of the best studied as well as Knapsack problem and various real problems are reformulated in meanings of these two approaches in order to apply known effective algorithm. In the classical bin-packing formulation, we are given a list of real numbers in $(0, 1]$ and the goal is to place them in a minimum number of bins so that no bin holds numbers summing to more than 1. The problem is extremely important in practice and finds numerous applications in scheduling, routing and resource allocation problems. Theoretically the problem has rich connections with discrepancy theory, iterative methods, entropy rounding and has led to the development of several algorithmic techniques. There is also a vast literature on mathematical models and exact algorithms for bin-packing.

Bin-packing is a special case of the one-dimensional cutting stock problem Gilmore and Gomory (1961), loading problem Eilon and Christofides (1971) and several related scheduling problems Conway et al. (2003). In 1961 Gilmore and Gomory introduced, for this class of problems, the concept of column generation, by deriving it from earlier ideas of Ford Jr and Fulkerson (1958) and Dantzig and Wolfe

(1960). At the moment, for more than 60 years, a fairly large number of not only different algorithms, but also productions have accumulated Christensen et al. (2017), Delorme et al. (2016). Also, an interesting approach is to use the key ideas of algorithms for other problems, including the greedy algorithm for the knapsack Akçay et al. (2007), an improved nature inspired Abdel-Basset et al. (2018b), new Branch-and-Price-and-Cut algorithm Wei et al. (2020) and the flower pollination algorithm Abdel-Basset et al. (2018a).

Most of the recent articles are not aimed at increasing the cargo parameters in question, but at increasing to the three-dimensional bin-packing problem. For example, in Gzara et al. (2020) the column generation approach is considered. An algorithm for the placement of layers based on their density and carrying capacity limits is developed. The research Erbayrak et al. (2021) is to present a mathematical model for a load-balanced 3D bin-packing problem in which element orientation is allowed. We first propose a model with a complete set of constraints concerning 3D replacement, stability and orientation, and a set of target functions minimizing the number of hoppers used, the unity of the product family, and the deviation from the ideal baricenter.

In this paper the following practical bin-packing problem is under consideration. There is a set of goods which must be delivered. There are several types of goods and in each cargo unit only one type of goods can be placed. The maximum number of cargo units is known. In fact it is

[★] The reported study was funded by RFBR, Sirius University of Science and Technology, JSC Russian Railways and Educational Fund “Talent and success”, project 20-38-51010.

possible to use not all cargo units in a solution. Due to this fact it is possible to calculate the volume of undelivered products which is denoted by B_1 . If any item was not delivered it is obligated to pay penalty. The unit penalty is fixed and that is defined as α coefficient. The total penalty for all undelivered goods is calculated as αB_1 .

Each item of goods is separable or inseparable, that means it is possible to place certain item at different cargo units if it is necessary. However there is a fee for each division. All products are not identical, so the weight coefficients of their disunion are not equal. Denote by B_2 total weight quantity of divisions in a solution obtained. Also the unit costs of each disunion is constant for each goods subset and it is represented by β coefficient. As a result the total fee for provided disunion is expressed by the expression βB_2 .

Due to bin-packing formulation we want to minimize the number of cargo units in use for delivery. That means we want to minimize free space in these cargo units. Not always it is feasible to fill each cargo unit fully, so the summary volume B_3 of free space in all cargo units in solution was introduced. This part multiplied by γ coefficient represents the amount of lost profit.

In result the following Objective Function was constructed: $\alpha \cdot B_1 + \beta \cdot B_2 + \gamma \cdot B_3 \rightarrow \min$. This Objective function represents the minimizing total delivery costs. It should be noted that number of cargo units in use do not appear directly in the objective function, but the total weight costs of free space assume this.

Problem is strongly NP-hard because it is a generalization of the well-known (one-dimensional) bin-packing problem, in which a set of n positive values w_j has to be partitioned into the minimum number of subsets so that the total value in each subset does not exceed a given bin capacity W .

2. PROBLEM DEFINITION AND DATA GENERATION

The set of n products is given. Each one is characterized by weight w_j , type t_j and divisibility characteristic d_j . The maximum number of cargo units (bins) is m , the capacity of which is characterized by the type of items placed in it. That is, after placing an object of type t_k into a bin, an arbitrary bin obtains a determined capacity of W_{t_k} and cannot contain objects of any other type. The problem is to distribute all items at least partially into cargo units in such a way that the constraints are met with the objective function set before. In addition the L_j was introduced for each item j as a cost coefficient of any division bins.

The objective function consists of three components. The B_1 component is responsible for the number of unallocated weights of items. It is calculated as the difference between the total weight of the items and the used ones. The B_2 component is responsible for the number of cuts of the object and penalizes it by the product of the number of cuts by the weight of this object. The B_3 component is introduced when there is unused space in the bins.

Further the variables and constraints of the model arising at this problem are described. First of all it is necessary to introduce the decision variables p_{ij} , that is amount of

item j which was placed into a bin i . There are additional binary variables l_{ij} which represents if certain product j was placed into a bin i at least partly:

$$l_{ij} = \begin{cases} 1 & \text{if item } j \text{ is distributed to bin } i, \\ 0 & \text{otherwise.} \end{cases}$$

All decision variables were listed and the objective function may be expressed as following:

$$F = \alpha \cdot \left(\sum_{j=1}^n w_j - \sum_{j=1}^n \sum_{i=1}^m p_{ij} \right) + \beta \cdot \sum_{j=1}^n \left(L_j \cdot \sum_{i=1}^m l_{ij} \right) + \gamma \cdot \left(W - \sum_{j=1}^n \sum_{i=1}^m p_{ij} \right) \rightarrow \min.$$

The objective function consists of three components as it was presented before. The first component $B_1 = \sum_{j=1}^n w_j - \sum_{j=1}^n \sum_{i=1}^m p_{ij}$ is calculated as difference between total products volume and distributed ones. The second part $B_2 = \sum_{j=1}^n (L_j \cdot \sum_{i=1}^m l_{ij})$ represents penalty for products division. For each product j it is possible to calculate the number of parts on which the product was divided. After that this number is multiplied by certain coefficient L_j in order to take into account the difficulty of product disunion. The last one part $B_3 = W - \sum_{j=1}^n \sum_{i=1}^m p_{ij}$ represents the volume of free space in used bins. The part W is calculated as total bins capacity with regard to product types assigned to each of them.

Several constraints arise in this practical problem:

- That is impossible to place more product than its volume:

$$0 \leq \sum_{i=1}^m p_{ij} \leq w_j \quad j \in \{1, \dots, n\}.$$

- Due to unification of item types in each bin only products (or their parts) of the same type can be placed at certain bin:

$$\sum_{k \in T_q} l_{ik} = \sum_{j=1}^n l_{ij} \quad \forall t \in \{1, \dots, t\}, i \in [1, m].$$

- Any bin has certain capacity for certain product type. Denote by W_k set of all products of type k . Then volume of distributed products (or their parts) at particular bin is less then capacity for this type:

$$\sum_{j \in T_k} p_{ij} \leq W_k \quad \forall t \in \{1, \dots, t\}, i \in [1, m].$$

- Some products are inseparable, so the can be placed only fully into a bin:

$$d_j \cdot p_{ij} = d_j \cdot w_j \cdot l_{ij} \quad \forall j \in \{1, \dots, n\}, \forall i \in \{1, \dots, m\}.$$

The test instances were generated in the amount of 168 ones, which were solved optimally. The values of generating parameters were the following:

- $m \in [8; 1000]$ – maximum number of bins;
- $n \in [10; 1500]$ – number of products;
- $w_j \in [1; 150]$ – volume of a product;
- $t_j \in [1; 40]$ – type of a product;
- $d_j \in \{0, 1\}$ – divisibility of a product ($d_j = 1$ if a product is inseparable);

- $L_j \in [0.15w_j; 0.85w_j]$ – characteristic of a product division;
- $W_k \in [10; 100]$ – capacity of bins for product type k .

For the union penalties the constraint $\alpha > \beta > \gamma$ is set. We have chosen such values: $\alpha = 15$, $\beta = 7$, $\gamma = 3$. The problem was defined and the constraints were described. At the next section two effective heuristics are presented for solving this problem.

3. ALGORITHMS

In opposite to classical Bin-Packing problem in the practical problem defined at this paper there are two additional characteristics of products: type and divisibility. Two algorithms presented below based on different ideas. While Bin-Spraying Heuristic (BSH) deals mainly with divisibility characteristic, the Dense Packing Heuristic (DPH) operates with different product types in turn. In this section pseudocodes and descriptions of the algorithms are presented.

3.1 Bin-Spraying Heuristic

We are starting to fill the bins. To do this, each type is considered in order. If there are inseparable items in the type under consideration, then we put them in the bin in descending order. Thus, we are primarily interested in inseparable items, since the problem can be divided into two parts, the first of which is filling the space of bins with inseparable items, and the second part is filling with separable items.

An important clarification is that we are looking at whether there is enough space after placing an inseparable item to the bin for another inseparable item of the same type completely to the same bin. If this doesn't happen, then we move on to the next type.

In the next step, we distribute the separable items in descending order in the freest bin, so that the total size of these items on one bin doesn't exceed its size

By now, our algorithm has partially filled the number of bins equal to the number of item types. Now there are also completely free bins available. On the first free bin, we distribute the inseparable items of the largest type and so on until the inseparable ones run out.

We add the remaining items to the remaining free bins also in the order of indivisibility and divisibility, so that the condition on types is met and the total capacity does not exceed the capacity of the bin.

At the very last step of the algorithm, we are obliged to check whether each item has been distributed at least partially. If this condition was not met, we take the largest inseparable item of this type and change one unit of loading to one unit of loading of an unallocated item.

Algorithm 1 BSH

```

1: for each type do
2:   items in decrease order
3:   for bin in bins do
4:     for items_of_type in items_of_type do
5:       if can assign full item on considered bin
6:       then
7:         assign, refresh bin capacity
8:       end if
9:     end for
10:   items in decrease order
11:   for bin in type bins do
12:     for items_of_type in items_of_type do
13:       if remain bin capacity more then zero
14:       then
15:         assign to remain capacity, refresh
16:       end if
17:     end for
18:   end for
19: remain items capacity of type in decrease order
20: for bin in nontype bins do
21:   for items_of_type do
22:     if item more then bin then
23:       assign item to remain capacity of bin
24:       refresh bin capacity
25:       refresh items of this type capacity
26:     end if
27:     if item less then bin then
28:       assign item to capacity of bin
29:       refresh bin capacity
30:       refresh items of this type capacity
31:     end if
32:   end for
33: end for
34: for type of item do
35:   for cut items do
36:     assign maximum from items to bins of this type
37:     with the biggest remain capacity, refresh
38:   end for
39: for item in items do
40:   if item not assigned at all then
41:     change with maximum assigned item of this
42:     type
43:   end if
44: end for

```

3.2 Dense Packing Heuristic

The DPH algorithm is a two-step algorithm. First you need to find some initial non-random solution, and then you need to improve it. The initial solution is sought using a greedy algorithm. In turn, items from each type are distributed among bins. Then this solution is improved. Accordingly, if all the items have already been distributed at the first stage, then the second stage is no longer needed. If not all items have been distributed, then at the second stage we begin to redistribute them from one bin to another.

Algorithm 2 DPH algorithm

```

1: for each item type do
2:   Sort items in decreasing order
3: end for
4: while there is free bin do
5:   Choose type for allocation
6:   for all items of chosen type do
7:     if there is free space then
8:       Place the item to the bin fully
9:     else if the item is inseparable then
10:      if the "next" bin exist then
11:        Allocate the item to the bin
12:      end if
13:    else
14:      while there is "next" bin or item is not
        allocated fully do
15:        Place the item to the bin as much as
        possible
16:        if The "current" item is not allocated
        fully and the "next" bin exists then
17:          Set allocated workload on the "cur-
            rent" bin equal to zero
18:        end if
19:      end while
20:    end if
21:  end for
22: end while
23: while there are unallocated items do
24:   Choose item type with maximum total inseparable
    workload
25:   Choose item type for reallocation items
26:   Choose the bin with the minimal allocated work-
    load
27:   for all items of the "current" type do
28:     if the "current" item is inseparable then
29:       Allocate the "current" item to the bin fully
30:     else
31:       Allocate the "current" item to the free space
        of the bin
32:     end if
33:   end for
34:   if all items of this type allocated at least partly
    then
35:     Remove the "current" type
36:   end if
37:   for all "reuse" items do
38:     Find a bin which processes items of the same
    type
39:     Place the item on this bin
40:   end for
41: end while

```

The first idea is that we must choose type of items which are allocated at maximum quantity of bins. After that we should choose the bin with the smallest workload. The items from this bin we should reallocate at free space on other bins (which processed items of the same type) in certain ratio. As a result we have fast algorithm which provide appropriate solutions. About 50 percent of all test instances were solved optimally. Most of other solved instances have a very small relative error. In fact there is a

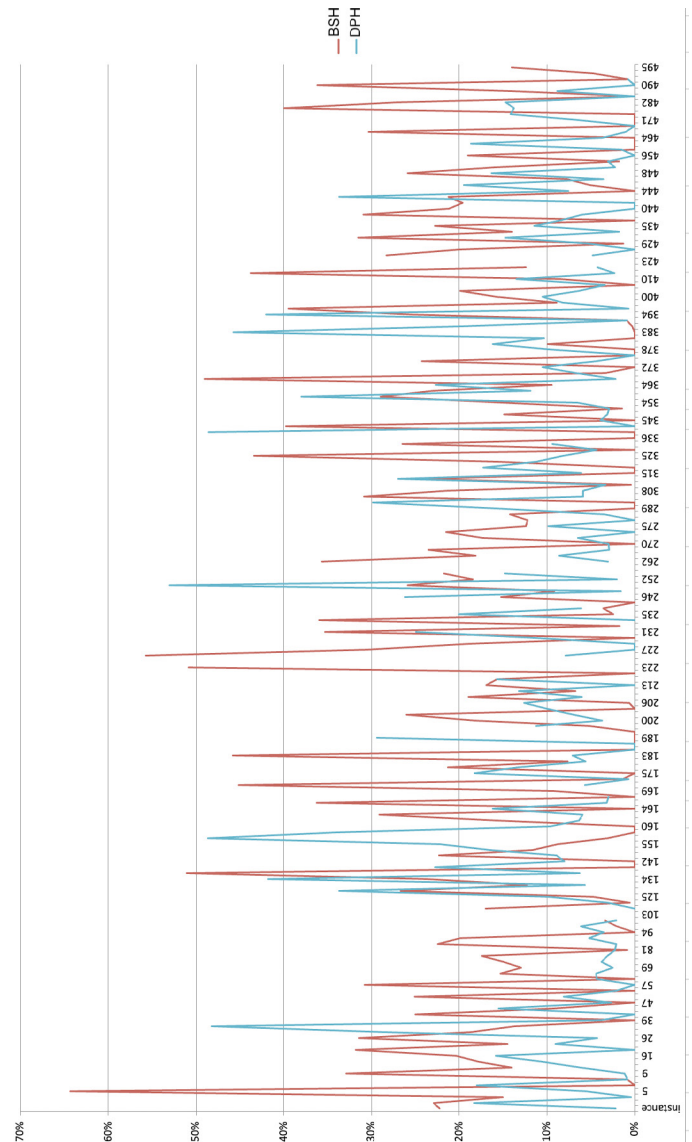


Fig. 1. Comparison of algorithm errors on solutions calculated by the exact solver.

difficulty with such a solution. Approximately 30 instances were solved incorrectly.

At the second stage of the algorithm in reallocation procedure mistakes may arise. In case if we reallocate inseparable item sometimes such an item may be divided (according to ratio defined). To fix this situation another requirements for choosing bin to reallocation were tested. The best option (with not very large computational complexity) is the following. As well as at previous method we choose a bin from the maximum list of bins that process items of the same type. Then we reallocate items from the bin one by one starting with the biggest inseparable item. For inseparable items we are looking for free space on corresponding bins or try to reduce the smallest separable items on each bin individually. After all inseparable items were reallocated we may reallocate separable items according to ratio defined. After updating the second stage of the bin algorithm only 12 test instances were solved incorrectly. Obviously it is possible to upgrade the algorithm to solve all instances in correct way, but it will take too

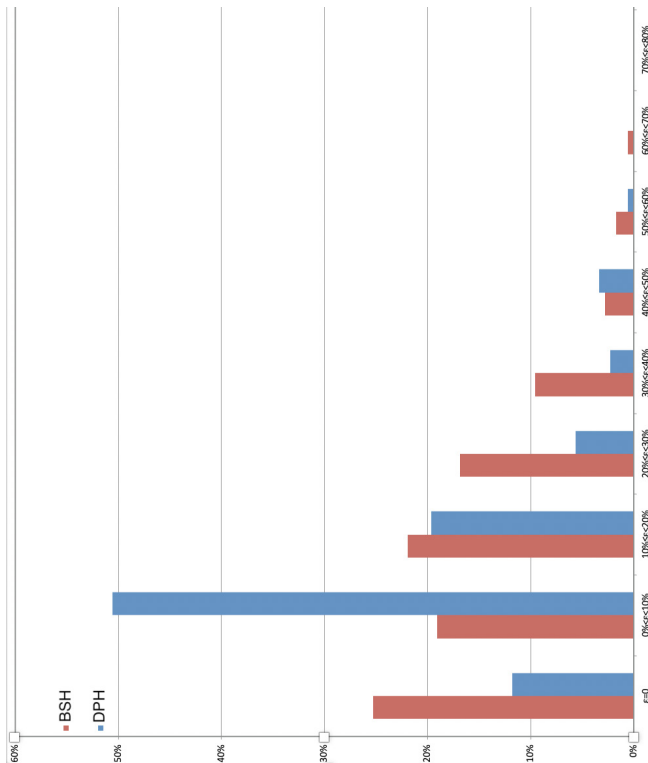


Fig. 2. The number of instances solved in a given error range.

much computational operations. However we are going to improve the algorithm or to combine it with other one with the aim to solve all instances as optimal and fast as it is possible.

The above algorithms have the main difference in that in the case of BSH, which is clear from the name, our items are distributed mainly across all the bins given to us, and then the remaining space is replenished. In the case of DPH, we try to occupy as few bins as possible for our items, thereby tightening the load. Depending on the scenario and the formulation of the problem, an algorithm is chosen that is better to solve it. If we need to use as few bins as possible, and the penalty for using an extra bin is high enough, then we need to use DPH. In the case when we need to distribute the cargo over a given number of bins more evenly, for example, in order to make it easier to unload it or we need bunkers of the same type to be nearby, then BSH is better suited. These algorithms are compared with the optimal solution obtained using the Gurobi solver.

4. RESULTS

The accuracy value of the algorithms was defined as the value of the relative error of the objective function relative to the exact solution. The exact solution was obtained using 168 instances from 100-500 bins. For each algorithm, the average value of the relative error, the minimum and maximum values of the relative error were calculated. The minimum error value turned out to be zero, since solutions matching the exact ones were obtained for each algorithm. The percentage of such instances for each algorithm was calculated.

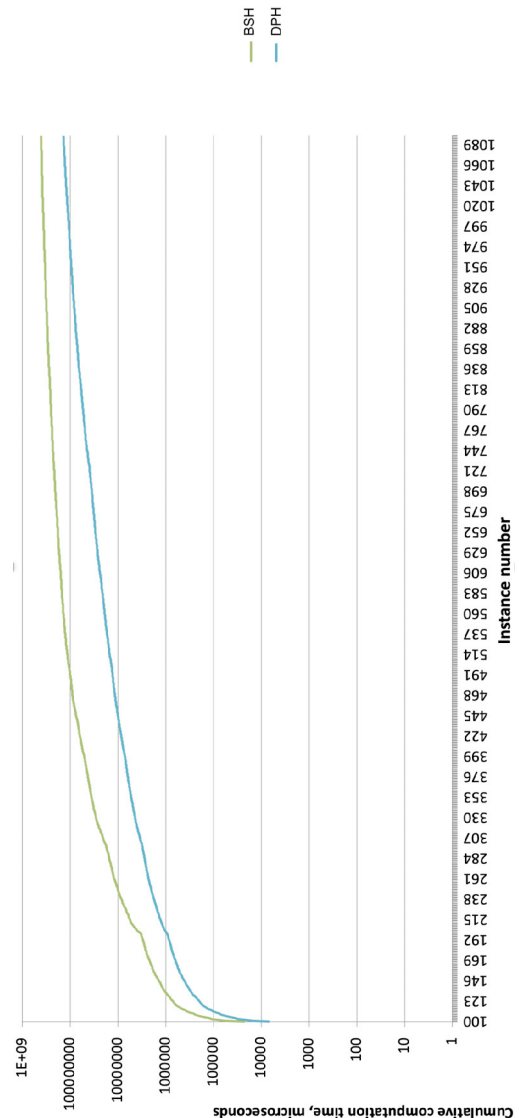


Fig. 3. Comparison of algorithm errors on solutions calculated by the exact solver.

In Figure 1, we can see that the algorithms give good results with alternating success due to the different generation of instances. Nevertheless, they almost all fit into an error equal to 50% for the entered objective function. In the Figure 2, you can see that the percentage of exact solutions obtained using the BSH algorithm exceeds almost 2 times the percentage of exact solutions of the DPH algorithm. But in the following columns we see that DPH is more stable for various instances, since it solves more than half of them with an error of no more than 10%.

Table 1. Experiment results.

Algorithm	BSH	DPH
The average error	14%	10%
The maximum error	64%	53%
Optimal instances	25%	13%

As for comparing the speed of these algorithms on instances with a large number of bins: DPH algorithm has significantly lower speed on average, but in the best cases its speed is much worse than the speeds of the other

two algorithms. In the worst cases, the algorithms show approximately the same speed value.

Table 2. Speed results.

Algorithm	BSH	DPH
Average speed	406315	137729
Maximum speed	998829	935215
Minimum speed	68	4043

5. CONCLUSION

This paper presented a completely new formulation of the well-known bin-packing problem. The work touches on such aspects of wagon formation as optimal loading, reduction of residual cargo, and product partitioning. Two algorithms are presented which allow the problem to be solved in a relatively fast time with a good percentage of error. The algorithms were tested on generated data that approximated real data, and were compared with the exact solution on a number of instances. The results suggest that the proposed algorithms are a good alternative to solving the proposed problem in adequate time. Also, the algorithms were compared with each other on very large instances and were found features of the initial data, which can be used to judge which of the algorithms is better.

REFERENCES

- Abdel-Basset, M., El-Shahat, D., El-Henawy, I., and Sangai, A.K. (2018a). A modified flower pollination algorithm for the multidimensional knapsack problem: human-centric decision making. *Soft Computing*, 22(13), 4221–4239.
- Abdel-Basset, M., Manogaran, G., Abdel-Fatah, L., and Mirjalili, S. (2018b). An improved nature inspired meta-heuristic algorithm for 1-d bin packing problems. *Personal and Ubiquitous Computing*, 22(5), 1117–1132.
- Akçay, Y., Li, H., and Xu, S.H. (2007). Greedy algorithm for the general multidimensional knapsack problem. *Annals of Operations Research*, 150(1), 17–29.
- Christensen, H.I., Khan, A., Pokutta, S., and Tetali, P. (2017). Approximation and online algorithms for multidimensional bin packing: A survey. *Computer Science Review*, 24, 63–79.
- Conway, R.W., Maxwell, W.L., and Miller, L.W. (2003). *Theory of scheduling*. Courier Corporation.
- Dantzig, G.B. and Wolfe, P. (1960). Decomposition principle for linear programs. *Operations research*, 8(1), 101–111.
- Delorme, M., Iori, M., and Martello, S. (2016). Bin packing and cutting stock problems: Mathematical models and exact algorithms. *European Journal of Operational Research*, 255(1), 1–20.
- Eilon, S. and Christofides, N. (1971). The loading problem. *Management Science*, 17(5), 259–268.
- Erbayrak, S., Özkır, V., and Yıldırım, U.M. (2021). Multi-objective 3d bin packing problem with load balance and product family concerns. *Computers & Industrial Engineering*, 159, 107518.
- Ford Jr, L.R. and Fulkerson, D.R. (1958). A suggested computation for maximal multi-commodity network flows. *Management Science*, 5(1), 97–101.
- Gilmore, P.C. and Gomory, R.E. (1961). A linear programming approach to the cutting-stock problem. *Operations research*, 9(6), 849–859.
- Gzara, F., Elhedhli, S., and Yildiz, B.C. (2020). The pallet loading problem: Three-dimensional bin packing with practical constraints. *European Journal of Operational Research*, 287(3), 1062–1074.
- Wei, L., Luo, Z., Baldacci, R., and Lim, A. (2020). A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems. *INFORMS Journal on Computing*, 32(2), 428–443.